

Programmation C

11 – Gestion des fichiers

Alain CROUZIL, Emmanuel LAVINAL

`alain.crouzil@irit.fr, emmanuel.lavinal@irit.fr`

Département d'Informatique
Faculté Sciences et Ingénierie
Université de Toulouse

Licence
Semestre pair 2025-2026

 | Département
Informatique

 Faculté
sciences et
ingénierie
Université
de Toulouse

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Introduction

Deux types de fichiers

Tous les fichiers sont des suites d'octets.

- **Fichiers de texte** : chaque octet qui les compose représente un caractère et ces caractères sont organisés en lignes (le contenu du fichier apparaît de manière lisible dans la fenêtre d'un éditeur de texte).
- **Fichiers binaires** : ce sont tous les autres fichiers (leur contenu apparaît sous la forme de caractères illisibles ou partiellement lisibles dans la fenêtre d'un éditeur).

Différence entre les deux types de fichiers : manière dont les informations y sont codées.

Exemple : l'entier 24 est codé par :

- 00110010 00110100 dans un fichier de texte : un octet pour le code ASCII du caractère 2, qui vaut 50, et un octet pour le code ASCII du caractère 4, qui vaut 52 ;
- 00000000 00000000 00000000 00011000 dans un fichier binaire si les entiers sont codés sur 4 octets.

Sommaire

- 1 Introduction
- 2 Bibliothèque standard**
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Entrées-sorties dans les fichiers

Utilisation de la bibliothèque standard

- Nécessité de la directive `#include <stdio.h>`.
- En général, on n'utilise pas les mêmes fonctions pour réaliser des entrées-sorties dans les fichiers de texte et dans les fichiers binaires.

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture**
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Ouverture (1/2)

Identificateur de fichier

La manipulation d'un fichier va se faire à travers un identificateur de fichier (aussi appelé *flux* ou *stream*) de type `FILE *`.

Fonction `fopen`

`FILE *fopen(const char *nom, const char *mode)`

ouvre le fichier de désignation `nom` dans le mode `mode` et retourne son identificateur ou `NULL` s'il y a un problème.

Le `mode` est une chaîne de 1, 2 ou 3 caractères :

- le premier caractère indique si le fichier doit être ouvert en lecture (`r` pour *read*), en écriture (`w` pour *write*) ou en écriture en fin de fichier (`a` pour *append*) ;
- le deuxième caractère, qui peut être ajouté au premier, est le signe `+` qui précise que l'on souhaite effectuer une mise à jour (lecture et écriture) ;
- le troisième caractère indique si le fichier doit être considéré comme un fichier de texte (`t` pour *text*), qui est le type par défaut, ou comme un fichier binaire (`b` pour *binary*).

Ouverture (2/2)

Exemple

```
FILE *id_fich;  
id_fich=fopen("data.txt","rt");
```

Unités standard

Trois identificateurs de fichiers particuliers sont disponibles et n'ont besoin ni d'être ouverts, ni d'être fermés : `stdin` (entrée standard), `stdout` (sortie standard) et `stderr` (sortie standard des erreurs).

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture**
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Lecture (1/4)

Lecture dans un fichier de texte

- **Lecture d'un caractère :**

```
int fgetc(FILE *id_fich)
```

- lit le caractère courant dans le fichier d'identificateur `id_fich`;
- retourne ce caractère ou EOF si la fin du fichier est atteinte.

Remarque : la macro `getc` effectue la même chose que `fgetc` et la macro `getchar` effectue la même chose que `fgetc(stdin)`.

- **Lecture d'une chaîne de caractères :**

```
char *fgets(char *ch, int n, FILE *id_fich)
```

- lit au maximum $n-1$ caractères dans le fichier d'identificateur `id_fich`, s'arrête si elle rencontre le caractère `'\n'`, les range (y compris l'éventuel `'\n'`) dans la chaîne d'adresse `ch` en complétant par le caractère `'\0'` ;
- retourne l'adresse de la chaîne ou `NULL` s'il y a un problème ou si la fin de fichier a été atteinte.

- **Lecture formatée :**

```
int fscanf(FILE *id_fich, const char *format[, liste_d_expr])
```

se comporte de la même manière que `scanf` mais effectue la lecture dans le fichier d'identificateur `id_fich` au lieu du clavier.

Lecture (2/4)

Lecture dans un fichier de texte : exemple

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      /* Ouverture du fichier */
6      FILE *f; /* Identificateur (interne) de fichier */
7      f=fopen("fich1.txt","rt"); /* r : lecture, t : texte */
8
9      /* Lectures dans le fichier */
10     char c1,c2,ch[81];
11     int n;
12     c1=fgetc(f); /* c1 recoit 'a' */
13     c2=fgetc(f); /* c2 recoit '\n' */
14     fgets(ch,81,f); /* ch recoit "bc d\n" */
15     fscanf(f,"%d",&n); /* n recoit 3794 */
16
17     /* Fermeture du fichier */
18     fclose(f);
19
20     printf("%c%c%s%d\n",c1,c2,ch,n);
21     return 0;
22 }
```

fich1.txt

a
bc_d
3794

Lecture (3/4)

Accès à la taille d'un emplacement en mémoire

`sizeof type`

L'opérateur `sizeof` permet de connaître la taille en octets occupée par une variable d'un type donné. La valeur obtenue est de type `size_t`.

Lecture dans un fichier binaire

```
size_t fread(void *pt, size_t taille, size_t nb_infos,  
             FILE *id_fich)
```

- lit dans le fichier d'identificateur `id_fich` `nb_infos` informations (blocs d'octets) de `taille` octets chacune et les range en mémoire à l'adresse `pt` (la zone mémoire doit avoir été préalablement allouée) ;
- retourne le nombre d'informations effectivement lues. Ce nombre peut être inférieur à `nb_infos` si la fin du fichier est atteinte.

Lecture (4/4)

Lecture dans un fichier binaire : exemple

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      /* Ouverture du fichier */
6      FILE *f; /* Identificateur (interne) de fichier */
7      f=fopen("fich1.dat","rb"); /* r : lecture, b : binaire */
8
9      /* Lecture dans le fichier */
10     float tab[100];
11     int n;
12     n=fread(tab,sizeof(float),100,f);
13
14     /* Fermeture du fichier */
15     fclose(f);
16
17     for (int i=0; i<n; i++) printf("%f\n",tab[i]);
18     return 0;
19 }
```

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture**
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Écriture (1/5)

Écriture dans un fichier de texte

• Écriture d'un caractère :

```
int fputc(int c, FILE *id_fich)
```

- écrit dans le fichier d'identificateur *id_fich* le caractère *c* (convertit en unsigned char);
- retourne le caractère écrit ou EOF s'il y a un problème.

• Écriture d'une chaîne de caractères :

```
int fputs(const char *ch, FILE *id_fich)
```

- écrit dans le fichier d'identificateur *id_fich* la chaîne d'adresse *ch*;
- retourne EOF s'il y a un problème ou une valeur non négative dans le cas contraire.

• Écriture formatée :

```
int fprintf(FILE *id_fich, const char *format[, liste_d_expr])
```

se comporte de la même manière que `printf` mais effectue l'écriture dans le fichier d'identificateur *id_fich* au lieu de l'écran.

Écriture (2/5)

Écriture dans un fichier de texte : exemple

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     /* Ouverture du fichier */
6     FILE *f; /* Identificateur (interne) de fichier */
7     f=fopen("fich2.txt","wt"); /* w : écriture, t : texte */
8
9     /* Écritures dans le fichier */
10    fputc(' z ',f);
11    fputc('\n',f);
12    fprintf(f,"Entier_:_%d\nFlottant_:_%f\n",12,3.14);
13    fputs("Chaine\n",f);
14
15    /* Fermeture du fichier */
16    fclose(f);
17
18    return 0;
19 }
```

fich2.txt

```
z
Entier_:_12
Flottant_:_3.140000
Chaine
```

Écriture (3/5)

Affichage des messages d'erreur

- Avec la fonction `fprintf` :

```
FILE *f;  
f=fopen("toto.txt","rt");  
if (f==NULL)  
    fprintf(stderr,"Fichier_toto.txt_inexistant.\n");
```

- Avec la fonction `perror` :

```
void perror(const char *message)
```

permet d'afficher sur `stderr` la chaîne passée en paramètre suivie du caractère `:`, suivi éventuellement du message associé à l'erreur (dépend du système d'exploitation).

```
FILE *f;  
f=fopen("fich","rt");  
if (f==NULL)  
    perror("fich");
```

Si le fichier `fich` n'existe pas, `perror` va afficher un message ressemblant à :
`fich: No such file or directory`

Écriture (4/5)

Écriture dans un fichier binaire

```
size_t fwrite(void *pt, size_t taille, size_t nb_infos,  
              FILE *id_fich)
```

- écrit dans le fichier d'identificateur *id_fich* *nb_infos* informations (blocs d'octets) de *taille* octets chacune situées en mémoire à l'adresse *pt* ;
- retourne le nombre d'informations effectivement écrites ; ce nombre peut être inférieur à *nb_infos* dans le cas d'un disque saturé.

Écriture (5/5)

Écriture dans un fichier binaire : exemple

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      /* Ouverture du fichier */
6      FILE *f; /* Identificateur (interne) de fichier */
7      f=fopen("fich2.dat", "wb"); /* w : écriture, b : binaire */
8
9      /* Écriture dans le fichier */
10     int tab[5]={1,2,3,4,5};
11     fwrite(tab, sizeof(int), 5, f);
12
13     /* Fermeture du fichier */
14     fclose(f);
15
16     return 0;
17 }
```

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture**
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils

Fermeture

Fermeture d'un fichier ouvert

```
int fclose(FILE *id_fich)
```

- ferme le fichier d'identificateur *id_fich*;
- retourne 0 ou EOF s'il y a un problème.

Remarques

- Il ne faut fermer un fichier que s'il est ouvert.
- Il est conseillé de fermer un fichier dès que son traitement est terminé.

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante**
- 8 Suppression d'un fichier
- 9 Conseils

Gestion de la position courante (1/4)

« Pointeur de fichier »

- À chaque fichier ouvert est associé un « pointeur de fichier » qui donne la position courante dans le fichier, c'est-à-dire le rang du prochain octet à lire ou à écrire.
- Après chaque opération de lecture ou d'écriture, ce « pointeur » est automatiquement incrémenté du nombre d'octets transférés.
- On parle d'*accès séquentiel* au fichier.

Détection de fin de fichier

```
int feof(FILE *id_fich)
```

- retourne une valeur non nulle si la fin du fichier d'identificateur *id_fich* est atteinte, ou 0 sinon ;
-  la valeur de retour de `feof` n'est valide qu'après avoir effectué au moins une lecture ;
- dans le cas d'une lecture dans un fichier binaire, la valeur de retour de la fonction `fread` peut remplacer avantageusement l'appel à la fonction `feof`.

Gestion de la position courante (2/4)

Positionnement dans un fichier

Il est possible d'accéder à un fichier de manière directe (accès direct) en modifiant sa position courante grâce à la fonction :

```
int fseek(FILE *id_fich, long decalage, int depart)
```

- *id_fich* est l'identificateur du fichier ;
- *decalage* est le nombre d'octets dont on veut se décaler par rapport à la position *depart* ;
- *depart* peut être égal à :
 - `SEEK_SET` ou 0 : début de fichier,
 - `SEEK_CUR` ou 1 : position courante dans le fichier,
 - `SEEK_END` ou 2 : fin de fichier.

La fonction `fseek` retourne 0 si tout s'est bien passé ou -1 sinon, en particulier dans le cas où on essaie de revenir en arrière avant le début du fichier. Néanmoins, quand on essaie d'avancer au-delà de la fin du fichier, `fseek` retourne 0 (tout se passe comme si on « piétinait » à la fin du fichier).

Gestion de la position courante (3/4)

Positionnement dans un fichier (suite)

- `void rewind(FILE *id_fich)`
place le pointeur du fichier d'identificateur *id_fich* à son début.
`rewind(f);` $\iff$ `fseek(f, 0, SEEK_SET);`
- `long ftell(FILE *id_fich)`
retourne la position courante du fichier d'identificateur *id_fich* ou `-1` s'il y a un problème.

Gestion de la position courante (4/4)

Exemples

On suppose que `f` est l'identificateur d'un fichier ouvert en lecture.

- **Mémorisation de la position courante :**

```
long position;  
position=ftell(f);
```

- **Retour à une position mémorisée :**

```
fseek(f,position,SEEK_SET);
```

- **Détermination de la taille d'un fichier :**

```
fseek(f,0,SEEK_END);  
long taille;  
taille=ftell(f);
```

- **Accès direct au contenu d'un fichier binaire :**

```
int Nieme(int n, FILE *f)  
{  
    fseek(f,sizeof(int)*(n-1),SEEK_SET);  
    int Entier;  
    fread(&Entier,sizeof(int),1,f);  
    return Entier;  
}
```

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier**
- 9 Conseils

Suppression d'un fichier

Fonction `remove`

```
int remove(const char *nom_fichier)
```

- supprime le fichier de nom *nom_fichier*;
- retourne 0 ou une valeur non nulle s'il y a un problème.

Sommaire

- 1 Introduction
- 2 Bibliothèque standard
- 3 Ouverture
- 4 Lecture
- 5 Écriture
- 6 Fermeture
- 7 Gestion de la position courante
- 8 Suppression d'un fichier
- 9 Conseils**

Conseils

Principales fonctions utilisées selon le type de fichier

<i>Type fichier</i>	Lecture			Écriture	
	<i>Mode ouverture</i>	<i>Fonctions</i>	<i>Détection fin de fichier</i>	<i>Mode ouverture</i>	<i>Fonctions</i>
Texte	"rt"	fgetc fgets fscanf	feof	"wt" ou "at"	fputc fputs fprintf
Binaire	"rb"	fread	valeur de retour de fread	"wb" ou "ab"	fwrite

QCM 11.1 – 11.2



Exercice

Exercice 11.1 Comptage des mots d'un fichier de texte

Sans utiliser d'allocation dynamique, écrivez un programme qui affiche le nombre de mots contenus dans le fichier de texte `fich.txt`. Les mots sont séparés par un ou plusieurs caractères séparateurs. On suppose qu'un mot peut contenir jusqu'à 26 caractères.

Exercice 11.2 Comptage des mots d'un fichier de texte : version sécurisée

Sans utiliser d'allocation dynamique, modifier le programme précédent pour éviter une erreur d'accès à la mémoire si un mot du fichier contient un nombre de caractères supérieur au maximum théorique de 26. Dans ce cas, le nombre de mots affiché pourra être incorrect.

Exercice 11.3 Copie d'un fichier

Écrivez un programme C qui copie le contenu du fichier `fichier-source.dat` dans le fichier `fichier-destination.dat` par blocs de 512 octets. Si le fichier destination existe déjà, son contenu doit être « écrasé ».